- It uses a 192-bit per-message nonce, which minimizes the risk of accidental nonce reuse when using randomly generated values. This is the maximum size of nonce, so you can use a shorter value if you absolutely need to save space and pad it with zeroes before decrypting.
- The XSalsa20 cipher and Poly1305 MAC used by NaCl can be compactly implemented in software on a wide range of devices. They are particularly suited to 32-bit architectures, but there are also fast implementations for 8-bit microcontrollers. They therefore make a good choice on platforms without hardware AES support.
- The 128-bit authentication tag use by Poly1305 is a good trade-off between security and message expansion.

If your devices are capable of performing public key cryptography, then NaCl also provides convenient and efficient public key authenticated encryption in the form the `CryptoBox` class, shown in listing 12.15. The `CryptoBox` algorithm works a lot like COSE's ECDH-SS algorithm in that it performs a static key agreement between the two parties. Each party has their own key pair along with the public key of the other party (see section 12.4 for a discussion of key distribution). To encrypt, you use your own private key and the recipient's public key, and to decrypt the recipient uses their private key and your public key. This shows that even public key cryptography is not much more work when you use a well-designed library like NaCl.

> **WARNING** Unlike COSE's HKDF, the key derivation performed in NaCl's crypto box doesn't bind the